

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.axes import Axes
from matplotlib.figure import Figure
from matplotlib.transforms import Bbox
from matplotlib.widgets import Slider, Button, CheckButtons
from matplotlib.patches import Rectangle

def draw_delta(ax: Axes, axPos, height, deltacolor, labelText):
    ax.arrow(axPos, 0, 0, height, head_width=0.01*10000,
             head_length=3, length_includes_head=True, color=deltacolor, label=labelText)

```

```

def is_there_aliasing(fs, fb):
    if fs/2 < fb:
        return True
    else:
        return False

```

The parametrized function to be plotted

```

def get_sin_value(t, frequency, amplitude=1) -> any:
    return amplitude * np.sin(2 * np.pi * frequency * t)

```

```

def get_alising_frecuency(fb, fs) -> list:

```

```

    aliasFrec = []

    k = 1
    while(-10000 < fb - k*fs):
        aliasFrec.append(fb - k*fs)
        k = k + 1

    k = 1
    while(10000 > -fb + k*fs):
        aliasFrec.append(-fb + k*fs)
        k = k + 1

    return aliasFrec

```

The function to be called anytime a slider's value changes

```

def update(fig, axSine, axFreq, freq_slider, samp_slider, period_slider):

    deltaH = 20 # height delta in frequency

    axSine.clear()
    axFreq.clear()

    t = np.arange(0, period_slider * (1/freq_slider),
                  (period_slider * (1/freq_slider))/1000) # start, stop, step
    y = np.cos(t*freq_slider*2*np.pi)

    # Check for aliasing
    aliasFrec = get_alising_frecuency(freq_slider, samp_slider)
    if is_there_aliasing(samp_slider, freq_slider):

        # Drawing signal
        if freq_slider < samp_slider or freq_slider > samp_slider * 3/2:
            y_alias = np.cos(
                -t*np.min(np.abs(aliasFrec))*2*np.pi)
        else:
            y_alias = np.cos(
                t*np.min(np.abs(aliasFrec))*2*np.pi)
        axSine.plot(t, y_alias, '--', color='c',
                   label="aliasing %d [Hz]" % np.min(np.abs(aliasFrec)))

    # Drawing spectre
    draw_delta(axFreq, aliasFrec[0], deltaH, 'c', "aliasing freq.")
    for i in range(1, len(aliasFrec), 1):
        draw_delta(axFreq, aliasFrec[i], deltaH, 'c', None)

    t_samples = np.arange(0, period_slider * 1/freq_slider, 1/samp_slider)
    y_samples = np.cos(t_samples*freq_slider*2*np.pi)
    axSine.plot(t, y, color='m', label="base %d [Hz]" % freq_slider)
    axSine.stem(t_samples, y_samples, linefmt='--')

    # Drawing nquist frequency
    draw_delta(axFreq, samp_slider/2, deltaH, 'black', "nyquist freq.")
    draw_delta(axFreq, -samp_slider/2, deltaH, 'black', None)

    # Drawing sampling frequency

```

```

draw_delta(axFreq, samp_slider, deltaH, 'r', "sample freq.")
draw_delta(axFreq, -samp_slider, deltaH, 'r', None)
draw_delta(axFreq, freq_slider, deltaH, 'm',
            "base freq.")      # Drawing input frequency
draw_delta(axFreq, -freq_slider, deltaH, 'm',
            None)              # Drawing input frequency
axFreq.set_xlim([-10000, 10000])
axFreq.set_ylim([0, 30])
axFreq.set_xlabel("f")
axFreq.set_ylabel("AU")
axSine.set_xlabel("t")
axSine.set_ylabel("AU")

axFreq.legend()
axSine.legend()
axFreq.add_patch(Rectangle((-samp_slider/2, 26), 2*samp_slider/2, 30,
                           edgecolor='pink',
                           facecolor='pink'))
axFreq.text(0, 27.5, "LP", horizontalalignment='center',
            verticalalignment='center')
fig.canvas.draw_idle()

```

```

def main():

    # Define initial parameters
    fb = 1000 # freq base
    fs = 5000 # freq sampling

    # Make grid and add axes
    fig: Figure = plt.figure(0)
    axSine = fig.add_subplot(3, 1, 1)
    axFreq = fig.add_subplot(3, 1, 2)
    axFreqSlider = fig.add_axes([0.32, 0.25, 0.5, 0.03])
    axSampSlider = fig.add_axes([0.32, 0.20, 0.5, 0.03])
    axPeriodSlider = fig.add_axes([0.32, 0.15, 0.5, 0.03])
    fig.tight_layout(pad=2.0)

    # Generate sliders
    freq_slider = Slider(ax=axFreqSlider,
                        label="Base Signal Frequency",
                        valmin=500,
                        valmax=10000,
                        valinit=fb,
                        orientation="horizontal",
                        valstep=1,
                        valfmt="%d [Hz]"
                        )
    freq_slider.vline._linewidth = 0

    samp_slider = Slider(ax=axSampSlider,
                        label="Sampling Frequency",
                        valmin=500,
                        valmax=10000,
                        valinit=fs,
                        orientation="horizontal",
                        valstep=1,
                        valfmt="%d [Hz]"
                        )
    samp_slider.vline._linewidth = 0

    period_slider = Slider(ax=axPeriodSlider,
                        label="Periods Quantity",
                        valmin=1,
                        valmax=30,
                        valinit=4,
                        orientation="horizontal",
                        valstep=1,
                        valfmt="%d"
                        )
    period_slider.vline._linewidth = 0

    # # register the update function with each slider
    def lambFreq(x): return update(fig, axSine, axFreq,
                                   x, samp_slider.val, period_slider.val)

    def lambSample(x): return update(
        fig, axSine, axFreq, freq_slider.val, x, period_slider.val)

    def lambPeriod(x): return update(
        fig, axSine, axFreq, freq_slider.val, samp_slider.val, x)

    # Plot base and sampled senoid
    update(fig, axSine, axFreq, freq_slider.val,
           samp_slider.val, period_slider.val)

    freq_slider.on_changed(lambFreq)

```

```
samp_slider.on_changed(lambSample)
period_slider.on_changed(lambPeriod)
```

```
plt.show()
```

```
if __name__ == "__main__":
    main()
```